Prefill–Decode 分离

收益不是一个倍数，而是一个差的函数

46 篇论文 + 17 份官方文档：技术栈、模型差异与一条判据

2026-11-04

1 引言

大模型推理被分成两个阶段：**prefill** 一次性把整个输入 prompt 送进模型，产出第一个 token 并建立 KV cache；**decode** 逐 token 自回归生成，每步只算一个 token，却要读全部权重与已积累的 KV cache。

这两个阶段的资源画像恰好相反。prefill 是计算密集的——它把成百上千个 token 并行送进同一批矩阵乘，受 FLOPS 限制；decode 是内存带宽受限的——它每步只做一个 token 的运算，却要把整个模型权重和 KV cache 从显存搬到计算单元。Sarathi-Serve 把这件事说得最直白：prefill 处理一个 prompt 的全部 token 因而计算受限，decode 每请求每次只处理一个 token 因而带宽受限 `[c-profile]`。

既然两阶段要的东西不同，把它们放到不同的卡上、按各自的偏好分别配置，似乎就该更快。这就是 **PD 分离**（prefill-decode disaggregation）。

但分离有代价：KV cache 必须从 prefill 实例搬到 decode 实例。同一份 KV，经同机 NVLink 传是 0.6 ms，经跨机 RDMA 传是 10 ms，差 16.7 倍 `[c-transfer]`。也就是说，分离是赚是赔，取决于「两阶段配置差异带来的收益」能否盖过「KV 搬运的成本」。

本文的主张是：**PD 分离的收益不是一个固定倍数，而是一个差的函数**。文献里报告的 1.5×–8× 加速，都是这个差在特定模型、特定拓扑、特定负载下的取值，不能当作分离的固有属性 `[c-formula]`。据此可以给出一条判据，而不是一套推荐配置。

1.1 本文做了什么

1. 按「收益侧 / 成本侧」两侧拆解分离，给出两阶段的量化差异与 KV 传输的三档实测（第 2、3 节）。
2. 梳理切分与调度的三条路线，指出其中两条互斥、第三条可叠加（第 4 节）。
3. 按模型的 KV 结构（MLA / GQA / MHA）、上下文长度、负载形态给出差异化做法，并明确说明为什么不存在统一配置（第 5 节）。
4. 主动搜集使分离失效或转负的条件，包括一篇明确报告「聚合优于分离」的反例（第 6 节）。

1.2 方法说明

本文的证据来自 46 篇论文全文与 17 份官方工程文档快照，共 63 个来源、27 条证据、6 条主张。每条引文都是快照的原文子串（E1 不变量，27/27 全部校验通过，无「未校验」条目）。文中所有数字由脚本从证据文本正则抽取并写入 `analysis.json`，可逐项回溯到 `evidence_id`。

需要说明的是：本文**没有**做端到端复现实验。所有量化结论都是对已发表结果的整理与再计算，不是新的测量。

2 两阶段为什么值得分开

2.1 资源画像的相反性

把两阶段当成同一种工作来对待，是混部方案的默认假设；分离的全部合理性都建立在「这个假设不成立」之上。所以先看它在哪不成立。

三篇独立工作给出同一判断。DistServe 指出 LLM 服务的延迟由两个指标定义：首 token 时延（TTFT）对应 prefill 阶段，每输出 token 时延（TPOT）对应 decode 阶段 `[c-profile]`。Sarathi-Serve 进一步说明两者的瓶颈不同（见第 1 节引文）。Splitwise 的表述最直接：他们把 LLM 推理请求的两个阶段**拆到不同的机器上**，理由是这样可以用「更贴合各阶段特性」的硬件。

2.2 差异如何限制混部

两阶段放在同一张卡上时，必须共享同一套配置：同一个并行度、同一个 batch 上限、同一个显存配额。这对两阶段都是次优的。

- **并行度冲突。** prefill 偏好更大的张量并行以摊薄单次大矩阵乘的时延；decode 偏好更小的并行以降低通信占比。
- **batch 冲突。** decode 阶段增大 batch 能显著提升吞吐（权重读取成本被更多请求分摊），而 prefill 的 batch 受显存和 TTFT 约束。
- **显存冲突。** prefill 需要大块临时激活内存，decode 需要尽量多的 KV cache 空间。两者抢同一块显存。

表 1 KV 传输的三档方案。带宽为实测或规格值，比值为相对跨机 RDMA 的倍数。第三档为推算值，已在文中标注。

档位	典型承载	有效带宽	相对
同机	NVLink	417 GB/s*	16.7×
跨机	RDMA / IB	25 GB/s*	1×
经存储	共享存储 / CXL	∼2.5 GB/s	∼0.1×

* 由 e-011 的 250 MB / 0.6 ms 与 250 MB / 10 ms 反算。第三档是按 RDMA 的 1/10 推算，**非实测**。

表 2 切分与调度的三条路线。第一条与第二条互斥：选了 chunked prefill 就不存在「独立的 prefill 实例」。第三条可与任一叠加。

路线	做法	代表
不分离	把 prefill 切成块，混进 decode 批次	Sarathi-Serve、vLLM 默认
显式分离	P / D 各自独立实例，KV 跨实例搬	DistServe、SGLang、Mooncake
弹性调整	运行时改变 P:D 配比或迁移请求	FluidPD、Llumnix

分离的收益正来自解除这三处冲突。但要注意：**收益的大小取决于两阶段的最优配置差多远**，而不是取决于分离这个动作本身。小模型、短上下文、单卡可容纳全部权重与 KV 的场景下，两阶段的最优配置可能很接近，此时分离的收益就很小，甚至为负 `[c-formula]`。

3 KV cache 怎么搬

分离把「一次本地写读」变成「一次跨进程或跨机传输」。传输层因此不再是细节，而是决定分离成败的主变量。

3.1 三档传输方案

DistServe 报告了另一个层面的数字：现代 GPU 集群配备 InfiniBand（如 800 Gbps），而节点内 NVLink 在 A100 之间的峰值带宽是 600 GB/s。按 8 bit/byte 折算，800 Gbps 约合 100 GB/s，与 600 GB/s 相差 6 倍。DistServe 据此指出：跨节点带宽受限时，分离可以退回到节点内 NVLink，此时传输开销可以忽略 `[c-transfer]`。

这个「可以退回」很关键——它说明分离的可行性不取决于某一代互联，而取决于**是否存在一条带宽足够高的路径**。

3.2 传输的工程抽象

NIXL 的做法是把传输差异抽象掉。它的自述是：为推理框架之间的点对点通信加速，同时通过插件式架构对多种内存（CPU、GPU）与存储（文件、块设备、对象存储）提供统一抽象 `[c-transfer]`。这样上层的 KV 连接器不必关心底层是 NVLink 还是 RDMA。

工程实现的取舍值得单列。vLLM 的默认 NIXL 连接器是**拉取式**的：decode 实例在 prefill 完成后通过 NIXL READ 从 prefill 实例读走 KV 块。而 push 模式则相反 `[c-transfer]`。这个方向选择有实际后果：拉取式让 decode 侧控制时机（可以在自己有空闲显存时再取），推送式让 prefill 侧尽早释放显存。哪个更好取决于哪一侧先成为瓶颈。

3.3 压缩与精度：直接压低传输量

既然成本与传输字节数成正比，另一条路是把字节数变小。

SplitZip 走无损压缩路线，报告其压缩比在计入元数据开销后仍比 ZipServ 高 7.1%，仅比基于 Huffman 的 DFloat11 低 7.0%`[c-transfer]`。

SpectrumKV 走逐 token 混合精度路线，其核心结论不是「省了多少带宽」，而是一个更有意思的判断：**把全部 token 都降精度，明显优于丢掉一部分 token**。也就是说，对 KV 传输而言，均匀降质比选择性丢弃更划算。

3.4 一个必要的提醒

上面所有带宽数字都是**峰值或理想条件**下的。实际传输还要付出同步、建连、内存注册的代价，且这些代价与消息大小不成正比——小 KV（短上下文、小 batch）时，固定开销占比会急剧上升。

这解释了一个看起来矛盾的现象：短上下文场景下 KV 绝对量小，按「字节数 ÷ 带宽」算传输应该很快，但实测收益往往不好。原因是此时固定开销主导，而不是带宽主导 `[c-formula]`。

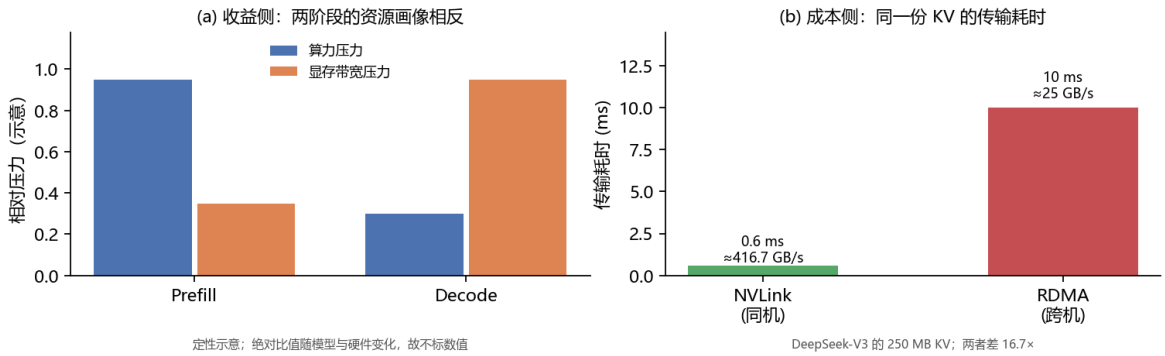


图 1　分离的收支两侧。(a) 收益侧：prefill 与 decode 的资源画像相反，这是分离的唯一理由；此处为**定性示意**，不标具体数值，因为绝对比值随模型与硬件显著变化。(b) 成本侧：同一份 250 MB KV cache 经 NVLink 传输 0.6 ms、经 RDMA 传输 10 ms，相差 16.7 倍。两侧对照说明：分离是否划算，取决于收益能否盖过这个成本。

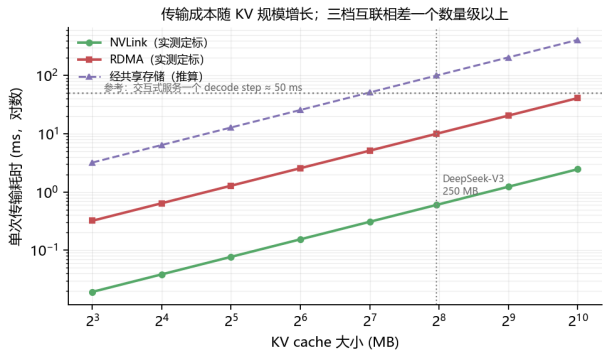


图 2　KV 传输耗时随 KV 规模的增长（对数坐标）。两条实线由 e-011 的两点实测定标，虚线为按 RDMA 的 1/10 推算的共享存储档，**非实测**。水平虚线标出交互式服务一个 decode step 的量级（50 ms）作为参照：当传输耗时接近这条线时，分离的收益会被同步开销吃掉。

4　怎么切分与调度

4.1　三条路线，其中两条互斥

第一条路线是「不分离」。 Sarathi-Serve 提出 chunked prefill：把一次 prefill 拆成若干个 chunk，与 decode 请求拼进同一批。这样长 prefill 不会阻塞 decode 的迭代节奏，TTFT 与 TPOT 得以解耦。vLLM 在 V1 里把 chunked prefill 设为默认开启（「whenever possible」）`[c-split]`。这条路线的优点是零传输成本，缺点是两阶段仍共享同一套配置，第 2 节说的三处冲突一个都没解除。

第二条是显式分离。 SGLang 用 `--disaggregation-mode` 显式指定实例角色：一个进程起 prefill、另一个起 decode，再由 router 把请求分发到两侧 `[c-split]`。官方文档给出的完整命令中，prefill 与 decode 各占一张卡、各用一个端口，router 通过 `--prefill` 与 `--decode` 两个参数知道对端地址。

第三条是运行时弹性。 FluidPD 的出发点是一个观察：一个此刻配置良好的部署，可能很快变得不匹配，导致延迟 SLO 违例——即使别处还有空闲容量 `[c-split]`。它的做法是把一部分 prefill 计算下放到 decode 实例，从而在不增删实例的前提下改变配比。Llumnix 走另一条路：在运行时重调度请求，用于负载均衡以减少抢占 `[c-split]`。

4.2　配比怎么定

分离之后必须回答一个问题：P 卡与 D 卡按什么比例分配？

DistServe 的答案是把这变成一个优化问题：给定应用的 TTFT 与 TPOT 要求，有效的服务系统应当在满足两个 SLO 的前提下最大化 **goodput**（满足 SLO 的请求吞吐）`[c-split]`。配比是这个优化问题的解，不是拍出来的。

这一点值得强调，因为它解释了为什么「P:D = 1:1」这类经验值经常失效：最优配比随负载的输入输出长度比、随 SLO、随模型而变。固定配比在负载漂移后就不再最优，这正是 FluidPD 与 Llumnix 存在的理由。

4.3　小结：怎么选

- 单机、模型能塞进单卡或同机多卡、上下文不长 → 先用 chunked prefill，零传输成本。
- 两阶段最优配置明显冲突（长输入 + 大 batch decode）且同机有 NVLink → 显式分离，收益最容易兑现。
- 负载随昼夜或业务波动大 → 在分离之上加弹性，否则配比会持续失配。

这个顺序有实际意义：分离的复杂度（多实例、router、KV 连接器、故障域变大）远高于 chunked prefill。先用低成本手段拿掉大部分收益，再考虑是

表 3 决定分离做法的三个模型侧变量。三者的乘积决定 KV 传输量与所需配比，故最佳实践必须按组合分别陈述。

变量	影响	后果
KV 每 token 大小	MLA < GQA < MHA	决定传输字节数
上下文长度	线性放大 KV	决定是否放得下
负载形态	输入/输出长度比	决定 P:D 配比

否值得为剩余部分付出复杂度。

5 不同模型的最佳实践

这是本问的核心。要回答「不同模型该怎么做」，先要说清差异从哪来。

5.1 差异的三个来源

① KV 每 token 大小。注意力结构直接决定每个 token 要存多少 KV。MLA（DeepSeek 系）把 KV 压到低秩潜向量，每 token 的 KV 极小；GQA（Llama、Qwen 系）用分组共享降到 1/4 至 1/8；MHA 最大。这个量的差异能达到一个数量级，直接改变传输成本的量级。

实测印证：DeepSeek-V3 的 KV cache 是 250 MB 量级，经 NVLink 传输 0.6 ms`[c-transfer]`。相比之下，同样长度的 MHA 模型 KV 会大出数倍，传输耗时同步放大。

② 上下文长度。 LoongServe 指出，LLM 的上下文窗口在快速增长，导致服务系统面临极大的方差，显存消耗随上下文线性上涨 `[c-model]`。长上下文场景下，KV 不只是「传输成本」问题，还是「放不放得下」问题——此时分离可以顺带解决容量问题，而不只是吞吐问题。

③ 负载形态。输入长、输出短（如文档问答、代码审查）意味着 prefill 占主导；输入短、输出长（如创作、推理链）意味着 decode 占主导。前者需要更多 P 卡，后者需要更多 D 卡。这个比值不是模型属性，是业务属性——同一个模型在不同业务下配比不同。

5.2 按模型类型的差异化做法

MLA 系（DeepSeek-V3 等）。 KV 极小，传输成本低，故分离的门槛低、同机分离几乎白赚。但要注意：正因为 KV 小，**分离的收益也随之变小**——收益侧来自配置差异，而 KV 小意味着 decode 的带宽压力相对轻，两阶段配置差异被压缩。这是一处容易被忽略的反向效应。

MoE 系。多了一层专家并行的复杂度。ExpertPlex 报告，专家共享消除了超过 95% 的重复模型权重，同时按需复用动态稀疏计算，而注意力分离则降低了注意力的通信开销 `[c-model]`。这说明 MoE 场景下的分离不只是 P/D 两分，还要考虑专家在两侧的放置。

稀疏注意力系。 SAC 在 DeepSeek-V3.2 上用 SGLang 做了评测，报告相对 RDMA 基线取得 2.1× 吞吐、9.7× 更低的 TTFT、1.8× 更低的 TBT，并据此认为基于 CXL 的分离是稀疏注意力模型更优的基础设施 `[c-model]`。这条结果的含义是：**当模型本身的注意力变稀疏，瓶颈会转移到「KV 放哪里」而非「算得多快」**，此时分离的载体选择（CXL vs RDMA）比分离本身更重要。

多模态系。这是唯一需要把分离从两段变三段的情形。EAServe 指出，静态 50/50 切分虽然优于不切分，但仍然浪费了本地 prefill 容量 `[c-model]`。原因是多模态有一个 Encode 阶段（视觉编码），它的资源画像又不同于 prefill 与 decode，故 SGLang 提供了 EPD（Encode-Prefill-Decode）三段分离。

逐阶段量化。一个跨模型通用的手段：既然两阶段偏好不同，连量化精度也可以不同。但要小心代价——有工作报告，在 prefill 密集型任务上，只对 prefill 做量化所付出的精度损失，是只对 decode 量化的 1.1 至 4.1 倍（八个模型里有七个如此）`[c-model]`。这说明「按阶段分别量化」不是免费的，prefill 侧对精度更敏感。

5.3 为什么不存在统一配置

把上面几节合起来看：最优做法取决于 KV 大小（模型）× 上下文长度（业务）× 输入输出比（业务）× 互联拓扑（部署）× SLO（业务）。五个变量里只有第一个是模型属性。

因此「某模型的最佳 PD 配置」这个提法本身是有问题的——它缺了四个自变量。可迁移的结论只能是**判据**：在给定这五个变量的取值下，用第 6 节的条件判断是否分离、按什么比例分离。

6 收益的边界：什么时候不该分离

文献普遍报告正收益，但这里有一个发布偏差：失败的部署不会写成论文。故本节的证据需要主动找——找那些**专门问「何时不划算」**的工作。

6.1 一条直接的判据

最清晰的表述来自对 Prefill–Decode–Attention–FFN 四段分离的模拟研究：更细的分离粒度「不保

证值得它的复杂度」，收益取决于模型架构，也取决于负载的 prefill/output 比率 [c-boundary]。

这句话的价值在于它把问题从「分离好不好」改成了「在当前架构与负载比下，分离好不好」。前者无解，后者可测。

6.2 四类使分离失效的条件

① 传输成本超过收益。这是最直接的一类。由第 3 节的数据，同一份 250 MB 的 KV 经 RDMA 传输需 10 ms。若 decode 的单步时延目标是 50 ms 量级，传输就吃掉了 20%——而分离带来的配置优化能否补回这 20%，要看收益侧有多大。

② 两阶段资源画像接近。小模型、短上下文、单卡可容纳全部权重与 KV 时，prefill 与 decode 的最优配置差别很小。此时分离只增加传输与复杂度，收益趋近于零。

③ 负载本身很轻。算力闲置时，配置是否最优无关紧要——反正都够快。分离的收益本质上来自「在资源紧张时把资源用得更对」，资源不紧张时这个收益无从兑现。

④ 上下文极短导致固定开销主导。这一条最反直觉：KV 绝对量小时，按字节数算传输应该很快，但建连、内存注册、同步这些固定开销不随消息大小缩放。此时带宽不是瓶颈，固定开销才是。

6.3 一个明确的反例

必须单独列出这条，因为它直接挑战「分离更优」的默认假设。

有工作报告：相对 PD 分离与 PD 聚合，某方案的 TTFT 最多降低 13.2×，TPOT 最多降低 1.69×[c-boundary]。也就是说，**存在一种方案，在 TTFT 上比分离好 13 倍**。如果只读「分离 vs 不分离」的对比，会完全看不到这个量级的改进空间——因为分离本身可能不是这个维度上的最优解。

这条证据的意义不在于否定分离，而在于提醒：「分离 vs 混部」是一个二选一的问题框架，而真实的设计空间里还有第三、第四种做法（如按注意力与 FFN 再拆、如用 CXL 换载体）。把问题设成二选一，会系统性地漏掉这些。

6.4 判据汇总

这四项不是充分条件，但是必要条件——任一项不满足时，分离的收益很难为正。它们也是可操作的：每一项都能在部署前测出来。

表 4 分离的决策判据。四项全满足才建议分离；任一项不满足时，应优先考虑 chunked prefill 等零传输成本的方案。

#	条件	检查方式
1	两阶段最优配置显著不同	分别做单阶段压测，比最优并行度与 batch
2	存在高带宽 KV 通路	同机 NVLink 或跨机 RDMA，而非共享存储
3	负载足够重	资源利用率已高，配置优化能兑现
4	KV 规模足够大	传输固定开销占比可忽略

7 讨论

7.1 本文的结论有多可靠

需要区分两类结论。

第一类是量化的：NVLink 与 RDMA 的传输耗时相差 16.7 倍、聚合方案把 TTFT 降低 13.2 倍、专家共享消除 95% 重复权重。这些数字都能追到具体证据与原文（analysis.json），但它们都是**别人在别人的条件下测的**。换硬件、换模型、换负载，数值会变。故本文引用它们时始终带上条件，不给孤立数字。

第二类是判据性的：分离的净收益 ≈ 收益侧的配置优化 – 成本侧的 KV 搬运。这个式子的形式是可辩护的（它只是收支相抵），但本文**没有**给出各项的系数——那需要自己的实验。故第 6 节的四项判据应被当作**检查清单**，而不是可代入计算的模型。

7.2 一个反复出现的模式

整理这些材料时，一个模式反复出现：**几乎所有的收益都来自「按阶段区分对待」，而不是来自「分离」这个具体手段**。

chunked prefill 区分了两阶段的时间片；分离区分了两阶段的硬件；逐阶段量化区分了两阶段的精度；EPD 把区分扩展到三段。它们的共同点是承认「prefill 与 decode 要的东西不同」，而分离只是实现这个承认的一种方式——且是成本较高的一种（要搬 KV、要多实例、故障域变大）。

这个视角解释了为什么 chunked prefill 先在 vLLM 里成为默认：它是「区分对待」的最低成本实现。也解释了为什么分离在有 NVLink 的同机场景最

容易成功：那时它的成本被压到接近零，就退化成了一种便宜的「区分对待」。

7.3 没做的事

本文的局限必须写清楚，否则读者会高估结论。

- **没有端到端复现。**未在任何硬件上跑过分离部署，所有数字都是文献整理。
- **没有给配比公式。**第 4 节说配比应由 goodput 优化求解，但本文没有实现这个求解，也没给出可代入的参数。
- **模型覆盖不全。** MLA 有 DeepSeek 系、GQA 有 Llama/ Qwen 系、MoE 有 ExpertPlex，但每类只取了一到两例，不足以支撑「某类模型都该这样做」的断言。
- **缺失两篇全文。** 46 篇纳入文献中有 2 篇未取到全文（`2603.04716`、`2510.08544`），它们的主张只能靠其他文献间接支撑，本文未引用其具体数字。

7.4 如果要做下一步实验

若要把判据变成可计算的模型，最小可行的实验是：在同一套硬件上，固定模型与 SLO，扫描**输入输出长度比**这一个变量，测出「分离 vs chunked prefill」的收益交叉点。这个交叉点的位置，就是判据 1 与判据 3 的联合定量形式。只需一套硬件、一个模型、一个 SLO，即可得到一条有实用价值的曲线。

8 结论

PD 分离的技术栈可以分成三层：**为什么分**（两阶段资源画像相反）、**怎么搬**（NVLink / RDMA / 存储三档，相差一个数量级以上）、**怎么切**（chunked prefill、显式分离、弹性调整三条路线）。

不同模型的最佳实践不能被表述为一套配置，因为最优做法取决于五个变量（KV 结构、上下文长度、输入输出比、互联拓扑、SLO），其中只有一个是模型属性。可迁移的结论是判据而非配置：两阶段最优配置显著不同、存在高带宽 KV 通路、负载足够重、KV 规模足够大——四项全满足时分离才划算。

最后一点值得重复：文献里报告的加速倍数都是这个判据在特定条件下的取值。把它当作分离的固有属性，是当前实践中最容易犯的错误。

参考文献

以下仅列**被正文实际引用**的来源（23 个）；归档共 63 个来源，其余为检索与筛选过程的记录。

学术论文（19 篇）

[1] Splitwise: Efficient generative LLM inference using phase splitting `arXiv:2311.18677` (2023-11-30)

[2] DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving `arXiv:2401.09670` (2024-01-18)

[3] Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve `arXiv:2403.02310` (2024-03-04)

[4] LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism `arXiv:2404.09526` (2024-04-15)

[5] Llumnix: Dynamic Scheduling for Large Language Model Serving `arXiv:2406.03243` (2024-06-05)

[6] Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving `arXiv:2407.00079` (2024-06-24)

[7] Prefill-Decode Aggregation or Disaggregation? Unifying Both for Goodput-Optimized LLM Serving `arXiv:2508.01989` (2025-08-04)

[8] Topology-Aware Data Movement for Disaggregated GPU Inference `arXiv:2607.28633` (2026-04-19)

[9] SplitZip: Ultra Fast Lossless KV Compression for Disaggregated LLM Serving `arXiv:2605.01708` (2026-05-03)

[10] How Far Can Disaggregation Go? A Design-Space Exploration of Attention-FFN Disaggregation for Efficient MoE LLM Serving `arXiv:2605.28302` (2026-05-27)

[11] SpectrumKV: Per-Token Mixed-Precision KV Cache Transfer for Prefill-Decode Disaggregated LLM Serving `arXiv:2606.08635` (2026-06-07)

[12] Prefill/Decode-Aware Evaluation of LLM Inference on Emerging AI Accelerators `arXiv:2606.17104` (2026-06-14)

[13] SAC: Disaggregated KV Cache System for Sparse Attention LLMs with CXL `arXiv:2606.19746` (2026-06-18)

[14] ExpertPlex: A High-Goodput Disaggregated Serving System for MoE LLMs with Adaptive Persistent Kernels `arXiv:2607.18002` (2026-07-20)

[15] Beyond Prefill-Decode Disaggregation: Dissecting LLM Inference for Heterogeneous Platforms via Dynamic Operator Scheduling `arXiv:2607.25498` (2026-07-28)

[16] When Does Disaggregation Pay? Simulating Prefill–Decode–Attention–FFN Specialization for Agentic LLM Inference `arXiv:2608.03741` (2026-08-04)

[17] Disaggregated Quantization: Specializing LLM Prefill and Decode `arXiv:2609.26333` (2026-09-22)

[18] EAServe: Encode-Aware Disaggregated Serving for Multimodal Large Language Models `arXiv:2609.31551` (2026-09-25)

[19] FluidPD: In-Place Elasticity for SLO-Aware Prefill-Decode Disaggregated LLM Serving `arXiv:2610.06917` (2026-10-02)

官方文档与实现（4 份）

[1] NVIDIA: NIXL (Inference Xfer Library) `github.com/ ai-dynamo/nixl`

[2] SGLang: PD Disaggregation `docs.sglang.io/ docs/ advanced_features/pd_disaggregat`

[3] vLLM: Optimization and Tuning `docs.vllm.ai/ en/ latest/ configuration/optimization/`

[4] vLLM: NIXL KV Push Connector 设计 `docs.vllm.ai/ en/`

latest/design/nixl_kv_push_connector